

Ex. No: 1

```
CREATE TABLE Student (StudentID int PRIMARY KEY, Name varchar(30), Age int);
```

Output:

Table created.

```
INSERT INTO Student VALUES (1, 'Alice', 20);  
INSERT INTO Student VALUES (2, 'Bob', 22);  
INSERT INTO Student VALUES (3, 'Charlie', 21);  
INSERT INTO Student VALUES (4, 'David', 19);
```

Output:

4 rows created.

```
CREATE TABLE Courses (CourseID int PRIMARY KEY, CourseName varchar(20));
```

Output:

Table created.

```
INSERT INTO Courses VALUES (101, 'Database Management');  
INSERT INTO Courses VALUES (102, 'Algorithms');  
INSERT INTO Courses VALUES (103, 'Web Development');
```

Output:

3 rows created.

```
CREATE TABLE Enrollments (StudentID int REFERENCES Student(StudentID), CourseID  
int REFERENCES Courses(CourseID));
```

Output:

Table created.

```
INSERT INTO Enrollments VALUES (1, 101);  
INSERT INTO Enrollments VALUES (1, 102);  
INSERT INTO Enrollments VALUES (2, 102);  
INSERT INTO Enrollments VALUES (3, 101);  
INSERT INTO Enrollments VALUES (3, 103);  
INSERT INTO Enrollments VALUES (4, 103);
```

Output:

6 rows created.

```
SELECT * FROM Student;
```

Output:

StudentID	Name	Age
1	Alice	20
2	Bob	22
3	Charlie	21
4	David	19

SELECT Name, Age FROM Student WHERE Age > 20;

Output:

Name	Age
Bob	22
Charlie	21

SELECT Name FROM Student WHERE StudentID IN (SELECT StudentID FROM Enrollments WHERE CourseID = (SELECT CourseID FROM Courses WHERE CourseName = 'Database Management'));

Output:

Name
Alice
Charlie

SELECT CourseID, CourseName FROM Courses WHERE CourseID IN (SELECT CourseID FROM Enrollments GROUP BY CourseID HAVING COUNT(*) > 1);

Output:

CourseID	CourseName
101	Database Management
102	Algorithms
103	Web Development

SELECT AVG(Age) AS AverageAge FROM Student;

Output:

AverageAge
20.5

SELECT Name, Age FROM Student WHERE Age > (SELECT AVG(Age) FROM Student);

Output:

Name	Age
Bob	22
Charlie	21

Ex. No: 2

```
CREATE TABLE Student (StudentID int, Name varchar(30), Age int);
```

Output:

Table created.

```
INSERT INTO Student VALUES (1, 'Alice', 20);  
INSERT INTO Student VALUES (2, 'Bob', 22);  
INSERT INTO Student VALUES (3, 'Charlie', 21);
```

Output:

3 rows created.

```
CREATE TABLE Courses (CourseID int, CourseName varchar(20));
```

Output:

Table created.

```
INSERT INTO Courses VALUES (101, 'Math');  
INSERT INTO Courses VALUES (102, 'English');  
INSERT INTO Courses VALUES (103, 'History');
```

Output:

3 rows created.

```
CREATE TABLE Enrollments (EnrollmentID int, StudentID int, CourseID int, Grade  
varchar(5));
```

Output:

Table created.

```
INSERT INTO Enrollments VALUES (1, 1, 101, 'A');  
INSERT INTO Enrollments VALUES (2, 1, 102, 'B');  
INSERT INTO Enrollments VALUES (3, 2, 101, 'A-');  
INSERT INTO Enrollments VALUES (4, 3, 103, 'B+');  
INSERT INTO Enrollments VALUES (5, 3, 102, 'A');
```

Output:

5 rows created.

INNER JOIN:

```
SELECT Student.StudentID, Student.Name, Student.Age, Courses.CourseID,  
Courses.CourseName, Enrollments.Grade FROM Student INNER JOIN Enrollments ON  
Student.StudentID = Enrollments.StudentID INNER JOIN Courses ON Enrollments.CourseID  
= Courses.CourseID;
```

Output:

StudentID	Name	Age	CourseID	CourseName	Grade
1	Alice	20	101	Math	A
1	Alice	20	102	English	B
2	Bob	22	101	Math	A-
3	Charlie	21	103	History	B+
3	Charlie	21	102	English	A

LEFT JOIN:

```
SELECT Student.StudentID, Student.Name, Student.Age, Courses.CourseID,  
Courses.CourseName, Enrollments.Grade FROM Student LEFT JOIN Enrollments ON  
Student.StudentID = Enrollments.StudentID LEFT JOIN Courses ON Enrollments.CourseID =  
Courses.CourseID;
```

Output:

StudentID	Name	Age	CourseID	CourseName	Grade
1	Alice	20	101	Math	A
1	Alice	20	102	English	B
2	Bob	22	101	Math	A-
3	Charlie	21	103	History	B+
3	Charlie	21	102	English	A

RIGHT JOIN:

```
SELECT Student.StudentID, Student.Name, Student.Age, Courses.CourseID,  
Courses.CourseName, Enrollments.Grade FROM Courses RIGHT JOIN Enrollments ON  
Courses.CourseID = Enrollments.CourseID RIGHT JOIN Student ON Enrollments.StudentID  
= Student.StudentID;
```

Output:

StudentID	Name	Age	CourseID	CourseName	Grade
1	Alice	20	101	Math	A
1	Alice	20	102	English	B
3	Charlie	21	103	History	B+
2	Bob	22	101	Math	A-
3	Charlie	21	102	English	A

Ex. No: 3

```
CREATE SEQUENCE student_seq START WITH 1 INCREMENT BY 1;
```

Output:

Sequence created.

```
CREATE TABLE students (student_id INT PRIMARY KEY, student_name VARCHAR(100),  
student_email VARCHAR(100));
```

Output:

Table created.

```
INSERT INTO students (student_id, student_name, student_email) VALUES  
(student_seq.NEXTVAL, 'Alice Johnson', 'alice@example.com');
```

```
INSERT INTO students (student_id, student_name, student_email) VALUES  
(student_seq.NEXTVAL, 'Bob Smith', 'bob@example.com');
```

```
INSERT INTO students (student_id, student_name, student_email) VALUES  
(student_seq.NEXTVAL, 'Charlie Brown', 'charlie@example.com');
```

Output:

3 rows created.

```
CREATE VIEW student_view AS SELECT student_id, student_name, student_email FROM  
students;
```

Output:

View created.

```
INSERT INTO students (student_id, student_name, student_email) VALUES  
(student_seq.NEXTVAL, 'Diana Prince', 'diana@example.com');
```

Output:

1 row created.

```
UPDATE students SET student_email = 'new_bob@example.com' WHERE student_name =  
'Bob Smith';
```

Output:

1 row updated.

```
DELETE FROM students WHERE student_name = 'Charlie Brown';
```

Output:

1 row deleted.

```
CREATE INDEX idx_student_email ON students (student_email);
```

Output:

Index created.

SELECT * FROM student_view;

Output:

student_id	student_name	student_email
1	Alice Johnson	alice@example.com
2	Bob Smith	new_bob@example.com
4	Diana Prince	diana@example.com

Ex. No: 4

```
CREATE TABLE customers (id NUMBER PRIMARY KEY, name VARCHAR2(30), age  
NUMBER, address VARCHAR2(50), salary NUMBER(10, 2));
```

Output:

Table created.

```
INSERT INTO customers VALUES (1, 'Ramesh', 32, 'Ahmadabad', 2000.00);  
INSERT INTO customers VALUES (2, 'Khilan', 25, 'Delhi', 1500.00);  
INSERT INTO customers VALUES (3, 'Kaushik', 23, 'Kota', 2000.00);  
INSERT INTO customers VALUES (4, 'Chaitali', 25, 'Mumbai', 6500.00);  
INSERT INTO customers VALUES (5, 'Hardik', 27, 'Bhopal', 8500.00);  
INSERT INTO customers VALUES (6, 'Komal', 22, 'MP', 4500.00);
```

Output:

6 rows created.

Implicit Cursor:

```
SET SERVEROUTPUT ON;  
DECLARE  
    total_rows number(2);  
BEGIN  
    UPDATE customers SET salary = salary + 500;  
    IF sql%notfound THEN  
        dbms_output.put_line('no customers selected');  
    ELSIF sql%found THEN  
        total_rows := sql%rowcount;  
        dbms_output.put_line(total_rows || ' customers selected ');  
    END IF;  
END;  
/
```

Output:

6 customers selected

PL/SQL procedure successfully completed.

```
SELECT * FROM customers;
```

Output:

ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmadabad	2500.00
2	Khilan	25	Delhi	2000.00
3	Kaushik	23	Kota	2500.00
4	Chaitali	25	Mumbai	7000.00
5	Hardik	27	Bhopal	9000.00
6	Komal	22	MP	5000.00

Explicit Cursor:

```
DECLARE
    c_id customers.id%type;
    c_name customers.name%type;
    c_addr customers.address%type;

    CURSOR c_customers IS SELECT id, name, address FROM customers;
BEGIN
    OPEN c_customers;
    LOOP
        FETCH c_customers INTO c_id, c_name, c_addr;
        EXIT WHEN c_customers%notfound;
        dbms_output.put_line(c_id || ' ' || c_name || ' ' || c_addr);
    END LOOP;
    CLOSE c_customers;
END;
/
```

Output:

PL/SQL procedure successfully completed.

ID	NAME	ADDRESS
1	Ramesh	Ahmadabad
2	Khilan	Delhi
3	Kaushik	Kota
4	Chaitali	Mumbai
5	Hardik	Bhopal
6	Komal	MP

Ex. No: 5

Procedure:

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE PROCEDURE Sum_Proc(a IN number, b IN number) IS
    c number;
BEGIN
    c := a + b;

    dbms_output.put_line('Sum of two nos = ' || c);
END Sum_Proc;
/
```

Output:

Procedure created.

```
DECLARE
    x number;
    y number;
BEGIN
    x := &x;
    y := &y;
    Sum_Proc(x, y);
END;
/
```

Output:

Enter value for x: 10
Enter value for y: 20
Sum of two nos = 30
PL/SQL procedure successfully completed.

Function:

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE FUNCTION Sum_Func(a IN number, b IN number) RETURN
number IS
    c number;
BEGIN
    c := a + b;

    RETURN c;
END Sum_Func;
/
```

Output:

Function created.

```
DECLARE
  no1 number;
  no2 number;
  result number;
BEGIN
  no1 := &no1;
  no2 := &no2;
  result := Sum_Func(no1, no2);
  dbms_output.put_line('Sum of two nos = ' || result);
END;
/
```

Output:

Enter value for no1: 5

Enter value for no2: 5

Sum of two nos = 10

PL/SQL procedure successfully completed.

Ex. No: 6

```
CREATE TABLE customer (sid NUMBER PRIMARY KEY, sname VARCHAR2(30), salary  
NUMBER);
```

Output:

Table created.

```
INSERT INTO customer VALUES (1, 'Aarthi', 300);  
INSERT INTO customer VALUES (2, 'Ramya', 500);  
INSERT INTO customer VALUES (3, 'Sakthi', 670);
```

Output:

3 rows created.

BEFORE UPDATE Trigger:

```
CREATE OR REPLACE TRIGGER up_classd  
BEFORE UPDATE ON customer  
FOR EACH ROW  
BEGIN  
    DBMS_OUTPUT.PUT_LINE('new value is ' || :new.salary);  
    DBMS_OUTPUT.PUT_LINE('old value is ' || :old.salary);  
END;  
/
```

Output:

Trigger created.

```
UPDATE customer SET salary = 500 WHERE sid = 1;
```

Output:

*new value is 500
old value is 300
1 row updated.*

```
SELECT * FROM customer;
```

Output:

SID	SNAME	SALARY
1	Aarthi	500
2	Ramya	500
3	Sakthi	670

BEFORE DELETE Trigger:

```
CREATE OR REPLACE TRIGGER del_classb
BEFORE DELETE ON customer
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('row deleted');
END;
/
```

Output:

Trigger created.

```
DELETE FROM customer WHERE sid = 1;
```

Output:

row deleted
1 row deleted.

```
SELECT * FROM customer;
```

Output:

SID	SNAME	SALARY
2	Ramya	500
3	Sakthi	670

```
CREATE TABLE classb (sid NUMBER, sname VARCHAR2(20), dept VARCHAR2(10),
stotal NUMBER, grade VARCHAR2(2));
```

Output:

Table created.

BEFORE INSERT Trigger (with user-defined exception):

```
CREATE OR REPLACE TRIGGER ins_classb
BEFORE INSERT ON classb
FOR EACH ROW
DECLARE
    InvTot EXCEPTION;
BEGIN
    IF :new.stotal > 1000 THEN
        RAISE InvTot;
    END IF;
EXCEPTION
```

```
        WHEN InvTot THEN
            RAISE_APPLICATION_ERROR(-20000, 'Total not valid');
    END;
/
```

Output:

Trigger created.

```
INSERT INTO classb VALUES (5, 'vino', 'it', 500, 'a');
```

Output:

1 row created.

```
INSERT INTO classb VALUES (6, 'jana', 'it', 20000, 'a');
```

Output:

ERROR at line 1:

ORA-20000: Total not valid

ORA-06512: at "SCOTT.INS_CLASSB", line 11

ORA-04088: error during execution of trigger 'SCOTT.INS_CLASSB'

Ex. No: 7

```
CREATE TABLE Authors (AuthorID INT PRIMARY KEY, FirstName VARCHAR2(50),  
LastName VARCHAR2(50));
```

Output:

Table created.

```
INSERT INTO Authors (AuthorID, FirstName, LastName) VALUES (1, 'George', 'Orwell');  
INSERT INTO Authors (AuthorID, FirstName, LastName) VALUES (2, 'Aldous', 'Huxley');  
INSERT INTO Authors (AuthorID, FirstName, LastName) VALUES (3, 'J.K.', 'Rowling');
```

Output:

3 rows created.

```
SELECT * FROM Authors;
```

Output:

AuthorID	FirstName	LastName
1	George	Orwell
2	Aldous	Huxley
3	J.K.	Rowling

```
CREATE TABLE Books (BookID INT PRIMARY KEY, Title VARCHAR2(100),  
Genre VARCHAR2(50), PublicationYear INT);
```

Output:

Table created.

```
INSERT INTO Books (BookID, Title, Genre, PublicationYear) VALUES (1, '1984',  
'Dystopian', 1949);  
INSERT INTO Books (BookID, Title, Genre, PublicationYear) VALUES (2, 'Brave New  
World', 'Dystopian', 1932);  
INSERT INTO Books (BookID, Title, Genre, PublicationYear) VALUES (3, 'Harry Potter and  
the Sorcerers Stone', 'Fantasy', 1997);
```

Output:

3 rows created.

```
SELECT * FROM Books;
```

Output:

BookID	Title	Genre	PublicationYear
1	1984	Dystopian	1949
2	Brave New World	Dystopian	1932
3	Harry Potter and the Sorcerers Stone	Fantasy	1997

```
CREATE TABLE BookAuthors (BookID INT,  
AuthorID INT,PRIMARY KEY (BookID, AuthorID),  
FOREIGN KEY (BookID) REFERENCES Books(BookID),  
FOREIGN KEY (AuthorID) REFERENCES Authors(AuthorID)  
);
```

Output:

Table created.

```
INSERT INTO BookAuthors (BookID, AuthorID) VALUES (1, 1);  
INSERT INTO BookAuthors (BookID, AuthorID) VALUES (2, 2);  
INSERT INTO BookAuthors (BookID, AuthorID) VALUES (3, 3);
```

Output:

3 rows created.

```
SELECT * FROM BookAuthors;
```

Output:

BookID	AuthorID
1	1
2	2
3	3

```
CREATE TABLE Borrowers (  
BorrowerID INT PRIMARY KEY,  
FirstName VARCHAR2(50),  
LastName VARCHAR2(50),  
MembershipDate DATE  
);
```

Output:

Table created.

```
INSERT INTO Borrowers (BorrowerID, FirstName, LastName, MembershipDate) VALUES  
(1, 'John', 'Doe', TO_DATE('2023-01-01', 'YYYY-MM-DD'));  
INSERT INTO Borrowers (BorrowerID, FirstName, LastName, MembershipDate) VALUES  
(2, 'Jane', 'Smith', TO_DATE('2023-02-15', 'YYYY-MM-DD'));
```

Output:

2 rows created.

```
SELECT * FROM Borrowers;
```

Output:

BorrowerID	FirstName	LastName	MembershipDate
1	John	Doe	2023-01-01
2	Jane	Smith	2023-02-15

```
CREATE TABLE BorrowedBooks (BorrowerID INT, BookID INT,  
    BorrowedDate DATE, ReturnDate DATE, PRIMARY KEY (BorrowerID, BookID),  
    FOREIGN KEY (BorrowerID) REFERENCES Borrowers(BorrowerID),  
    FOREIGN KEY (BookID) REFERENCES Books(BookID));
```

Output:

Table created.

```
INSERT INTO BorrowedBooks (BorrowerID, BookID, BorrowedDate, ReturnDate) VALUES  
(1, 1, TO_DATE('2023-03-01', 'YYYY-MM-DD'), TO_DATE('2023-03-15', 'YYYY-MM-DD'));  
INSERT INTO BorrowedBooks (BorrowerID, BookID, BorrowedDate, ReturnDate) VALUES  
(2, 3, TO_DATE('2023-03-05', 'YYYY-MM-DD'), TO_DATE('2023-03-20', 'YYYY-MM-DD'));
```

Output:

2 rows created.

```
SELECT * FROM BorrowedBooks;
```

Output:

BorrowerID	BookID	BorrowedDate	ReturnDate
1	1	2023-03-01	2023-03-15
2	3	2023-03-05	2023-03-20

Ex. No: 8

System-defined exception (NO_DATA_FOUND):

```
SET SERVEROUTPUT ON;
DECLARE
    c_id customers.id%type := 5;
    c_name customers.name%type;
    c_addr customers.address%type;
BEGIN
    SELECT name, address INTO c_name, c_addr FROM customers WHERE id = c_id;
    dbms_output.put_line('name: ' || c_name);
    dbms_output.put_line('address: ' || c_addr);
EXCEPTION
    WHEN no_data_found THEN
        dbms_output.put_line('no such customer!');
    WHEN others THEN
        dbms_output.put_line('error!');
END;
/
```

Output:

```
name: Hardik
address: Bhopal
PL/SQL procedure successfully completed.
```

User-defined exception:

```
DECLARE
    c_id customers.id%type := &cc_id;
    c_name customers.name%type;
    c_addr customers.address%type;
    ex_invalid_id EXCEPTION;
BEGIN
    IF c_id <= 0 THEN
        RAISE ex_invalid_id;
    ELSE
        SELECT name, address INTO c_name, c_addr FROM customers WHERE id = c_id;
        DBMS_OUTPUT.PUT_LINE('Name: ' || c_name);
        DBMS_OUTPUT.PUT_LINE('Address: ' || c_addr);
    END IF;
EXCEPTION
```

```
WHEN ex_invalid_id THEN
    dbms_output.put_line('ID must be greater than zero!');
WHEN no_data_found THEN
    dbms_output.put_line('No such customer!');
WHEN others THEN
    dbms_output.put_line('Error!');
END;
/
```

Output:

Enter value for cc_id: -6
ID must be greater than zero!
PL/SQL procedure successfully completed.

Ex. No: 10

```
use library;

db.authors.insertMany([
  { "AuthorID": 1, "FirstName": "George", "LastName": "Orwell" },
  { "AuthorID": 2, "FirstName": "Aldous", "LastName": "Huxley" },
  { "AuthorID": 3, "FirstName": "J.K.", "LastName": "Rowling" }
]);
```

Output:

```
{ "acknowledged": true, "insertedIds": [1, 2, 3] }
db.authors.updateOne({ "AuthorID": 1 }, { $set: { "LastName": "Smith" } });
```

Output:

```
{ "acknowledged": true, "matchedCount": 1, "modifiedCount": 1 }
db.authors.find().pretty();
```

Output:

```
{ "_id": ObjectId("..."), "AuthorID": 1, "FirstName": "George", "LastName": "Smith" }
{ "_id": ObjectId("..."), "AuthorID": 2, "FirstName": "Aldous", "LastName": "Huxley" }
{ "_id": ObjectId("..."), "AuthorID": 3, "FirstName": "J.K.", "LastName": "Rowling" }
db.authors.deleteOne({ "AuthorID": 1 });
```

Output:

```
{ "acknowledged": true, "deletedCount": 1 }
db.authors.find().pretty();
```

Output:

```
{ "_id": ObjectId("..."), "AuthorID": 2, "FirstName": "Aldous", "LastName": "Huxley" }
{ "_id": ObjectId("..."), "AuthorID": 3, "FirstName": "J.K.", "LastName": "Rowling" }
```

Ex. No: 11

```
CREATE KEYSPACE library WITH REPLICATION = { 'class': 'SimpleStrategy',  
'replication_factor': 3 };
```

Output:

Keyspace created.

```
USE library;
```

Output:

Context switched to library.

```
CREATE TABLE authors (  
  AuthorID int PRIMARY KEY,  
  FirstName text,  
  LastName text  
);
```

Output:

Table created.

```
INSERT INTO authors (AuthorID, FirstName, LastName) VALUES (1, 'George', 'Orwell');  
INSERT INTO authors (AuthorID, FirstName, LastName) VALUES (2, 'Aldous', 'Huxley');  
INSERT INTO authors (AuthorID, FirstName, LastName) VALUES (3, 'J.K.', 'Rowling');
```

Output:

3 rows inserted.

```
SELECT * FROM authors;
```

Output:

authorid	firstname	lastname
1	George	Orwell
2	Aldous	Huxley
3	J.K.	Rowling

```
UPDATE authors SET LastName = 'Smith' WHERE AuthorID = 1;  
SELECT * FROM authors WHERE AuthorID = 1;
```

Output:

authorid	firstname	lastname
1	George	Smith

```
DELETE FROM authors WHERE AuthorID = 1;
```

```
SELECT * FROM authors WHERE AuthorID = 1;
```

Output:

(Empty result set)